HARNESS LAB · MÓDULO 5 · AULA 5.1

O script *do loop*

Pega a fase, executa, roda a Prova, commita ou tenta de novo. Você não escreve mais o prompt de cada tarefa: escreve o laço que entrega o PHASES ao agente e deixa o teste e o avaliador decidirem.

`PHASES.md` `## Phase N:` `git status` `claude -p` `codex exec` `exit code` `TASK n: DONE` `ralph.sh` `loop.sh` `motor falso`

VOCÊ SAI COM

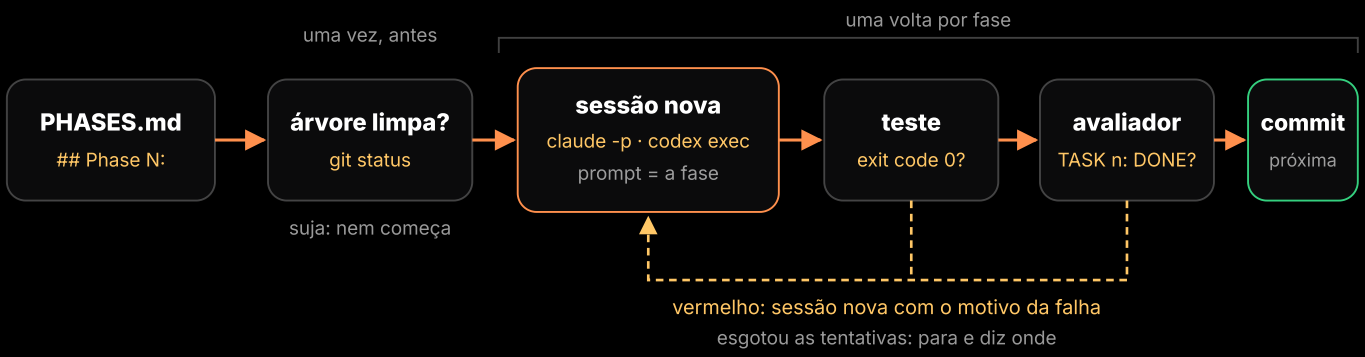
O seu núcleo de loop aprovado por um juiz que não gasta token, e o loop pronto do curso (o `ralph.sh` do BC Harness, chamado pelo `loop.sh`) instalado no seu projeto e ensaiado de ponta a ponta com um motor falso.

Material de apoio · Lucas Souza · Beer and Code

1 A aula em *uma página*

Com os artefatos do Módulo 4 prontos, a feature tem tudo que uma pessoa precisaria para desenvolver. Agora quem recebe é um agente, por um loop. "Não escrevo mais prompts, escrevo loops": o segredo é que o Módulo 4 já foi a escrita do loop. Aqui a gente monta o laço em cima dele.

O loop percorre o `PHASES.md`. O `/plan` da aula 4.3 confere o formato de forma determinística, então ele sempre sai igual e dá para quebrar em pedaços. Com 1 milhão de tokens de janela, a fase inteira cabe numa sessão; com janela menor, quebre por task. A linguagem é sua (shell, Python, PHP); o Lucas usa shell porque fala fácil com git e testes.



Árvore limpa antes

`git status` sujo e o loop nem começa: o que estava solto entraria no commit da fase. O dono decide: commit, stash ou apaga.

O agente headless

`claude -p` ou `codex exec`: executa sem interface e devolve o resultado. O prompt é a fase, mais papel e stack se quiser.

O teste decide

Quem roda a suíte é o loop: exit code 0 ou não, sem opinião de modelo. Dentro da sessão, o hook da aula 2.2 já devolve o erro (o `-p` roda os hooks do projeto); o gate roda a suíte de novo, fora dela.

O avaliador, o da prancheta

Outra sessão olha a fase, o `git status` e os arquivos alterados contra os critérios de aceite. Não fechou, a causa volta para quem implementa.

> *"`claude -p "prompt"` runs once and exits, which is how Claude Code ends up in CI jobs, git hooks, and shell loops."*
>
> Documentação oficial do Claude Code · Best practices, "Run Claude from a script" (code.claude.com/docs/en/best-practices)

Depois o Lucas abre o loop dele, o **ralph**: escolhe a engine (claude ou codex) e o modelo do avaliador, tem painel ao vivo, faz o gate da árvore suja, recebe o PHASES e a fase de partida e monta o prompt. O de implementação foi lido na aula: "Implemente COMPLETAMENTE a fase descrita abaixo", criando e rodando os testes listados, sem pular item marcado com `[ ]`. Falhou, entra o prompt de correção. A provocação do fim: **comece simples**. Quebre as fases; ponha o gate da árvore suja, a sessão headless com o prompt da fase, o teste e o avaliador, todos chamados pelo loop. O ralph que você instala, o do BC Harness 0.2.0, tem 1.903 linhas; o núcleo do material (seção 3) faz o laço em 39. Ou pegue o pronto.

2 Indo *além da aula*

ALÉM DA AULA · VERDE NÃO É PRONTO, E O "SÓ LEITURA" É PEDIDO

Caso real: "Tests: 178 passed" no gate 2 e "TASK 6: INCOMPLETE" no avaliador: faltava um limite de upload no nginx (lab-agent-seller, 12/07/2026). No claude, o avaliador do ralph usa `--model sonnet` (o da gravação, cópia de julho, haiku) e roda em bypass com `--allowedTools`, que só pré-aprova: o "não edite" é prompt. No seu loop, `--tools "Read,Glob,Grep"` com `--disallowedTools "mcp__*"` e sem bypass: a leitura é imposta pelo Claude Code (o `--tools` sozinho não tira o MCP).

Fonte: cli-reference, `--allowedTools` e `--tools`; `ralph.sh` :419-423, :1349-1353.

ALÉM DA AULA · BYPASS: O QUE AINDA SEGURA E QUEM PAGA

O ralph roda `claude --dangerously-skip-permissions -p`: na 2.1.302 em diante, um `-p` sem modo começa no Manual e nega o que pediria aprovação. A doc pede bypass só em ambiente isolado; mesmo assim, **deny** e **hooks** `PreToolUse` valem: as Travas do Módulo 2 seguram o loop. E no `-p` a `ANTHROPIC_API_KEY` "is always used when present": a conta vira API. O pré-voo do `loop.sh` barra essa e as outras credenciais.

Fonte: permission-modes; hooks-guide; authentication.

ALÉM DA AULA · O RALPH LEMBRA O QUE JÁ FEZ

As fases feitas ficam em `.phases/.progress`, valendo para o mesmo PHASES (sha256): rodou de novo, continua da primeira que falta; editou o PHASES, zera. A armadilha: depois de um ensaio no mesmo PHASES, o run de verdade pula tudo e diz "ENTREGOU 2/2" em 0 s, sem commit. Depois de ensaio, `rm -rf .phases`.

Fonte: `ralph.sh` :468-493; reproduzido com o motor falso.

ALÉM DA AULA · FASE OU TASK, E DUAS MECÂNICAS COM O MESMO NOME

Uma `## Phase N:` é uma sessão (`### Phase N.M` fica na da fase pai): uma task por sessão pede uma task por fase. Opus 4.7 e Sonnet 5 em diante têm 1M de janela. "Ralph is a Bash loop" (Geoffrey Huntley): o plugin `ralph-loop` repete o prompt na mesma sessão; o `ralph.sh` abre uma por fase.

Fonte: model-config, "Extended context"; `ralph.sh` :54, :1361; README do `ralph-loop`.

ALÉM DA AULA · USA O CODEX? E O MODELO?

O ralph troca de motor com `--engine codex` (o padrão dele): implementa com `codex exec --sandbox danger-full-access -` e avalia com `--sandbox read-only`, sandbox real. Chame o `ralph.sh` direto: o `loop.sh` é só claude, o shim usa flags dele. O `ENGINE` dos modelos só troca o binário: serve para CLI com as flags do claude, como o motor falso (no codex, `-p` é `--profile`). No Cursor: `agent -p --force`. Modelo pelo `loop.sh`: `RALPH_VERIFY_MODEL=haiku` no avaliador, `ANTHROPIC_MODEL` na implementação.

Fonte: `ralph.sh` :117, :1341-1343; `codex exec --help`; Cursor, CLI headless; model-config.

3 Instale e faça *no seu repositório*

1 Baixe os materiais do curso (uma vez só).

Terminal · na sua pasta pessoal — shell

```
cd ~
git clone https://github.com/beerandcodeteam/harness-lab-materiais
```

Já baixou antes? Atualize:

Terminal · em qualquer pasta — shell

```
git -C ~/harness-lab-materiais pull
```

2 O motor: o BC Harness, com o ralph.

É o plugin da aula 3.4. Já instalou? Os comandos não estragam nada e o `list` confirma `bc-harness@beer-and-code`.

Terminal — claude

```
claude plugin marketplace add beerandcodeteam/beer-and-code-harness
claude plugin install bc-harness@beer-and-code
claude plugin list
```

Ou, dentro do Claude Code:

Dentro do Claude Code — sessão

```
/plugin marketplace add beerandcodeteam/beer-and-code-harness
/plugin install bc-harness@beer-and-code
/reload-plugins
```

3 Confira que a sua máquina roda o ralph (zero token, cerca de 1 minuto).

É a suíte do próprio plugin, com engine falso. Esperado: a última linha fala em `187 asserts`, todos verdes. No macOS é aqui que aparece o que falta: o ralph pede bash 4 ou mais novo, `date` do GNU e `sha256sum`.

Terminal · em qualquer pasta — shell

```
C=~/.claude/plugins/cache/beer-and-code/bc-harness
bash "$(ls -d $C/*/scripts | sort -V | tail -n 1)/test-ralph.sh"
```

4 Monte o seu núcleo numa pasta de ensaio, sem gastar token.

Fora de qualquer repositório, com o juiz, o motor falso e os dois pedidos ao lado. O último comando roda o juiz no núcleo de referência: `testa-loop-minimo: 15 passaram`. É o placar que o seu precisa bater.

Terminal · na sua pasta pessoal — shell

```
A=~/harness-lab-materiais/modulo-5-loop/aula-5.1-o-script-do-loop
mkdir -p ~/harness-lab-sandbox
cd ~/harness-lab-sandbox
cp -r $A/modelos/teste $A/modelos/motor-falso $A/modelos/pedido-*.md .
bash teste/testa-loop-minimo.sh $A/modelos/loop-minimo.sh
```

Abra o Claude Code nessa pasta e cole o pedido. O texto também está no `pedido-nucleo.md` da pasta, junto com o que ler no script que ele escrever.

O PEDIDO DO NÚCLEO

Escreva loop-minimo.sh em bash, até 50 linhas. Uso: ./loop-minimo.sh PHASES.md. Variáveis: ENGINE (padrão claude), TESTE (obrigatório), TENTATIVAS (padrão 3). Para cada heading '## Phase N: título': sessão nova com `$ENGINE -p "<prompt>" --dangerously-skip-permissions < /dev/null`, com o texto da fase no prompt; qualquer outro heading '## ' encerra a fase, como no ralph. Rode $TESTE; verde → git add -A e commit 'feat(phase-N): título'; vermelho → nova tentativa com as últimas 40 linhas da saída sob 'Motivo da falha'. Esgotou → 'travou na fase N' e exit 1. Acabou → 'terminou' e exit 0.

Quando ele terminar, rode o juiz sem argumento: ele testa o `loop-minimo.sh` da pasta, o seu. Reprovou? Cole a falha no Claude Code e peça a correção uma vez.

Terminal · na pasta de ensaio — shell

```
cd ~/harness-lab-sandbox
bash teste/testa-loop-minimo.sh
```

5 **Os dois gates que faltam no núcleo: árvore limpa e avaliador.**

O pedido gera o `loop-com-gates.sh`: recusa árvore suja antes da primeira sessão e, com a Prova verde, chama um avaliador só com ferramentas de leitura, uma linha `TASK n` por task. O `grep` mostra o pedido, que está no `pedido-gates.md` da pasta com o que ler no resultado.

Terminal · na pasta de ensaio — shell

```
cd ~/harness-lab-sandbox
grep '^> ' pedido-gates.md
```

Cole no Claude Code, na mesma pasta, e rode o juiz dele. A referência faz `29 passaram`.

Terminal · na pasta de ensaio — shell

```
cd ~/harness-lab-sandbox
bash teste/testa-loop-com-gates.sh ./loop-com-gates.sh
```

6 **Instale o loop pronto no projeto, na branch da feature.**

O loop commita sozinho; na `main` o pré-voo recusa. Ainda na `main`? Crie a branch da feature, trocando `agenda-ics` pelo seu slug. Já está na da 4.3? Pule a segunda linha.

Terminal · na raiz do seu projeto shell

```
git branch --show-current
git switch -c feat/agenda-ics
```

O `instalar.sh` cria `.claude/scripts/` com o `loop.sh` e o que ele usa. Arquivo seu diferente fica intacto; o da aula vai para `.harness-lab-novo`. Os scripts entram no git; o log, não.

Terminal · na raiz do seu projeto shell

```
~/harness-lab-materiais/instalar.sh 5.1
git add .claude/scripts
git commit -m "harness: scripts do loop"
```

7

Pré-voo, zero token.

A primeira linha pega o PHASES mais recente; o `echo` mostra qual. O `confere-phases.sh` refaz a conferência de formato do `/plan`; o `--checa` roda o pré-voo e para. Esperado: `resultado: ok` e `pré-voo ok`. Cada `NÃO` diz o que corrigir (seção 5); ele cobra o hook dos testes de aceite da aula 4.2 (`instalar.sh 4.2`) e o seu `.claude/aceite.txt`.

Terminal · na raiz do seu projeto shell

```
P=$(ls -t .spec/features/*/PHASES.md | head -n 1)
echo "$P"
.claude/scripts/confere-phases.sh "$P"
.claude/scripts/loop.sh --checa "$P"
```

8

Ensaie o loop inteiro com o motor falso, numa branch descartável.

O ralph e a sua Prova de verdade, com o motor falso no lugar do `claude` (ele só escreve `src/fase-N.txt`). Esperado: `desfecho: ENTREGOU` e um commit `feat(phase-N)` por fase. Caiu no gate 2 na fase 1? A sua suíte já está vermelha antes do agente (o típico: teste de aceite sem o marcador de pendente).

Terminal · na raiz do seu projeto shell

```
A=~/harness-lab-materiais/modulo-5-loop/aula-5.1-o-script-do-loop
P=$(ls -t .spec/features/*/PHASES.md | head -n 1)
git switch -c ensaio-motor-falso
CLAUDE_REAL=$A/modelos/motor-falso/claude .claude/scripts/loop.sh "$P"
```

Volte e apague o ensaio. O `rm -rf .phases` não é opcional: sem ele o run de verdade pula as fases que o ensaio marcou como feitas (seção 2).

Terminal · na raiz do seu projeto shell

```
git switch -
git branch -D ensaio-motor-falso
rm -rf .phases
```

9 **O run de verdade (gasta token).**

Num terminal seu ou num tmux. O `--checa` mostrou o pior caso: fases × 3 ciclos × 2 sessões. O `loop.sh` já traz os freios da 5.2 (40 min e `--max-budget-usd 5` por sessão, estimativa do cliente que pode passar) e o aviso da 5.3. O curso faz o primeiro run pago no fim da 5.3, com uma task; quer ver antes, é este comando.

Terminal · na raiz do seu projeto — shell

```
P=$(ls -t .spec/features/*/PHASES.md | head -n 1)
.claude/scripts/loop.sh "$P"
```

O resumo fica em `.loop-logs/<run>/resumo.txt`: desfecho, fases feitas, onde parou, a causa e os commits.

O núcleo de referência *em 39 linhas*

A receita em código, sem avaliador e sem árvore limpa (esses estão no `loop-com-gates.sh`). Leia na ordem dos números.

modelos/loop-minimo.sh — no repositório · não copie do PDF

```
set -u
DOC="${1:?uso: TESTE=\"cmd\" ./loop-minimo.sh PHASES.md}"
ENGINE="${ENGINE:-claude}"
TESTE="${TESTE:?defina TESTE com o comando da Prova}"
TENTATIVAS="${TENTATIVAS:-3}"
TMP=$(mktemp -d)

# 1. Quebra as fases: um arquivo por "## Phase N: título"; outro "## " encerra a fase.
awk -v d="$TMP" '/^## Phase [0-9]+: /{n++; f=sprintf("%s/fase-%02d.md", d, n); print > f; next}
                 /^## /{f=""} f{print > f}' "$DOC"

for fase in "$TMP"/fase-*.md; do
  num=$(head -1 "$fase" | sed -E 's/^## Phase ([0-9]+):.*/\1/')
  titulo=$(head -1 "$fase" | sed -E 's/^## Phase [0-9]+: //')
  motivo=""
  for ((t = 1; t <= TENTATIVAS; t++)); do
    echo "fase $num, tentativa $t/$TENTATIVAS"
    # 2. Sessão nova, com a fase (e a causa da falha anterior) no prompt.
    prompt="Implemente a fase abaixo. Rode os testes antes de terminar.
$motivo
$(cat "$fase")"
    "$ENGINE" -p "$prompt" --dangerously-skip-permissions < /dev/null > "$TMP/sessao-$num-$t.log" 2>&1
    # 3. A Prova roda fora da sessão. 4. Verde: commit da fase.
    if bash -c "$TESTE" > "$TMP/prova.log" 2>&1 < /dev/null; then
      git add -A && { git diff --cached --quiet || git commit -qm "feat(phase-$num): $titulo"; }
      continue 2
    fi
    # 5. Vermelho: a causa real vai para a próxima tentativa.
    motivo="## Motivo da falha
$(tail -n 40 "$TMP/prova.log")"
  done
  # 6. Esgotou: para e diz onde.
  echo "travou na fase $num"
  exit 1
done
echo "terminou"
```

O loop.sh, na parte *desta aula*

O trilho em volta do ralph, sem editar nem copiar o ralph (ele fica no cache do plugin). Nesta aula: o pré-voo mínimo (git, PHASES, nenhuma credencial de API, ralph e `claude` achados, a Prova, sem a qual nem roda), a chamada `bash ralph.sh --engine claude --test-cmd "<Prova>" [--dashboard] PHASES.md` e o fim: copia o estado para `.loop-logs/<run>/` (com `cp -rp`, mantendo a hora dos logs) antes que o próximo run apague o `.phases/`, escreve o `resumo.txt` e sai com o exit code do ralph. Freios (5.2), vigia e aviso (5.3) já vêm no arquivo. Os comentários seguem as aulas: `[5.1]` é esta, `[5.2]` os freios, `[5.3]` o aviso.

4 Arquivos que *reforçam a aula*

`modelos/loop-minimo.sh e loop-com-gates.sh`

"Comece simples" em código: o núcleo em 39 linhas e a receita inteira da aula em 69, com a árvore limpa e um avaliador que só tem ferramentas de leitura.

No repositório: modulo-5-loop/aula-5.1-o-script-do-loop/modelos/

`modelos/teste/ e modelos/motor-falso/claude`

Os dois juízes e o `claude` falso: quem aprova o loop é um teste com exit code, não o modelo nem você. O mesmo motor roda o ralph de verdade no passo 8.

No repositório: modulo-5-loop/aula-5.1-o-script-do-loop/modelos/

`.claude/scripts/loop.sh e confere-phases.sh`

O loop pronto, que fecha as armadilhas de chamar o ralph na mão (engine codex, ralph na raiz sujando a árvore, chave de API), e a validação determinística do formato do PHASES que a aula cita.

No repositório: modulo-5-loop/aula-5.1-o-script-do-loop/projeto/

`modelos/pedido-nucleo.md e pedido-gates.md`

O "construa o seu" da aula: cada pedido diz o que ler no script que o Claude escrever (o `< /dev/null`, a Prova fora da sessão, o avaliador sem MCP) e como provar com o juiz.

No repositório: modulo-5-loop/aula-5.1-o-script-do-loop/modelos/

5 Erros *comuns*

SINTOMA	CAUSA	CORREÇÃO
"codex CLI nao encontrado", ou abre o codex	Ralph chamado sem `--engine claude`, como o `./ralph.sh` que o `/plan` imprime	Rode pelo `loop.sh`, que passa a engine
`NÃO árvore suja`	Arquivo sem commit entraria no commit da fase	Commit, `git stash -u` ou apague o que for descartável
`NÃO .diagnostico-operador está fora do git` (ou o `.md`)	O diagnóstico da aula 1.1 sem commit e sem exclude	Não apague: é o "antes" do curso. Commite, ou rode o `echo` do exclude que a mensagem mostra
`NÃO credencial de API`	Chave de API, Bedrock, Vertex, Foundry ou `apiKeyHelper`: no `-p` vence a assinatura	Tire do ambiente, ou `LOOP_ACEITO_API=1`

SINTOMA	CAUSA	CORREÇÃO
`NÃO ralph.sh ... não encontrado`	Plugin não instalado para o seu usuário	Passo 2, ou `RALPH_SH=caminho`
`NÃO hook guarda-aceite.py`	O hook dos testes de aceite falta ou não está registrado. É o da 4.2	`instalar.sh 4.2`. Só para ensaio: `LOOP_SEM_ACEITE=1`
`NÃO .claude/aceite.txt ausente ou vazio`	A 4.2 instala o `aceite.exemplo.txt`, não o `aceite.txt`	Copie para `.claude/aceite.txt` e liste os seus testes de aceite
"ENTREGOU" em 0 s, nenhum commit	Progresso de um ensaio no mesmo PHASES	`rm -rf .phases` e rode de novo
Juiz do núcleo reprova tudo no seu script com gates	Ele grava a saída dentro do repositório de teste: árvore suja	Use o `testa-loop-com-gates.sh`
`FALHOU a fase 2 não leva o '## Open Questions'`	O corte só fecha a fase no próximo `## Phase`	Qualquer outro `##` encerra a fase

6 Você sai *com*

- [ ] `test-ralph.sh` passa na sua máquina: `187 asserts`, todos verdes.
- [ ] O seu `loop-minimo.sh` faz `testa-loop-minimo: 15 passaram`; com os gates, `testa-loop-com-gates: 29 passaram`.
- [ ] `.claude/scripts/` commitado na branch da feature.
- [ ] `loop.sh --checa` termina em `pré-voo ok`, sem gastar token.
- [ ] O ensaio com o motor falso deu `desfecho: ENTREGOU`, um commit por fase, e o `.phases/` foi apagado depois.

PROMETIDO NA AULA

- ✓ **"Vai estar nos materiais de apoio para vocês pegarem esse loop já pronto e testar no projeto de vocês"**: o ralph vem no plugin BC Harness (passo 2); o `loop.sh` que chama ele, em `projeto/.claude/scripts/` (passo 6); o teste no seu projeto, passos 7 a 9.
- ✓ **"Pode pegar esse pronto que eu já vou deixar lá"**: `~/harness-lab-materiais/instalar.sh 5.1` e o README da aula no repositório.
- ✓ **"Ou construa o seu próprio"**: os pedidos `modelos/pedido-nucleo.md` e `modelos/pedido-gates.md`, as referências e os dois juízes em `modelos/` (passos 4 e 5). Para mostrar o seu, leve junto a saída do juiz.
- ✓ **"Vocês conseguem verificar isso"** (o formato do PHASES): o `/plan` confere antes de terminar, e o `confere-phases.sh` repete quando quiser (passo 7).
- ✓ **"Claude -p ou Codex Exec, que são os exemplos"**, **"engine claude, engine codex, tem um dashboard"** e **"modelos diferentes, você pode configurar"**: como trocar motor e modelo na seção 2 ("Usa o Codex? E o modelo?"); o `loop.sh` fixa o claude e liga o painel quando a saída é um terminal (`LOOP_PAINEL`).

7 Glossário

Headless	O agente sem interface: `claude -p` ou `codex exec`. Roda uma vez, devolve o resultado e sai.
Gate	Checagem que decide se a fase anda. Quem roda é o loop, não o agente. No log do ralph: a árvore suja é o pré-voo; por fase, gate 0 (a sessão terminou de verdade), gate 1 (escreveu algo, só sinal), gate 2 (a suíte) e gate 3 (o avaliador).
Avaliador	Sessão separada que confere task por task contra o critério de aceite. No ralph, gate 3.
Ciclo	Uma tentativa da fase. Vermelho abre o seguinte com o motivo; o padrão é 3.
Bypass	`--dangerously-skip-permissions`: nada pede aprovação. Deny e hooks ainda valem.
Motor falso	Um `claude` de mentira, em bash, que testa o encanamento sem gastar token.
ralph.sh e loop.sh	O motor do BC Harness, emprestado, e o trilho do curso em volta dele.

8 Para *ir além*

- Best practices: "Run Claude from a script", o loop em shell da própria doc, e "Let Claude run without you", as cinco perguntas antes de deixar rodar sozinho.
- Run Claude Code programmatically: o `-p`, os formatos de saída (`stream-json`, que o ralph lê) e o `--bare`, que vai virar padrão e desliga CLAUDE.md, hooks e o login da assinatura.
- BC Harness: o `ralph.sh`, o `ralph-watch.sh` do painel e o README com os quatro gates e as variáveis.
- Effective harnesses for long-running agents: a Anthropic chegou ao mesmo desenho, "one feature at a time", com estado em arquivo e git.

Próxima: *Aula 5.2 · Condições de parada: o freio do loop*

Ele desiste depois de três tentativas. E se uma sessão nunca termina? E quem paga?